



Presentación 5

PROBLEMAS DE CLASIFICACIÓN

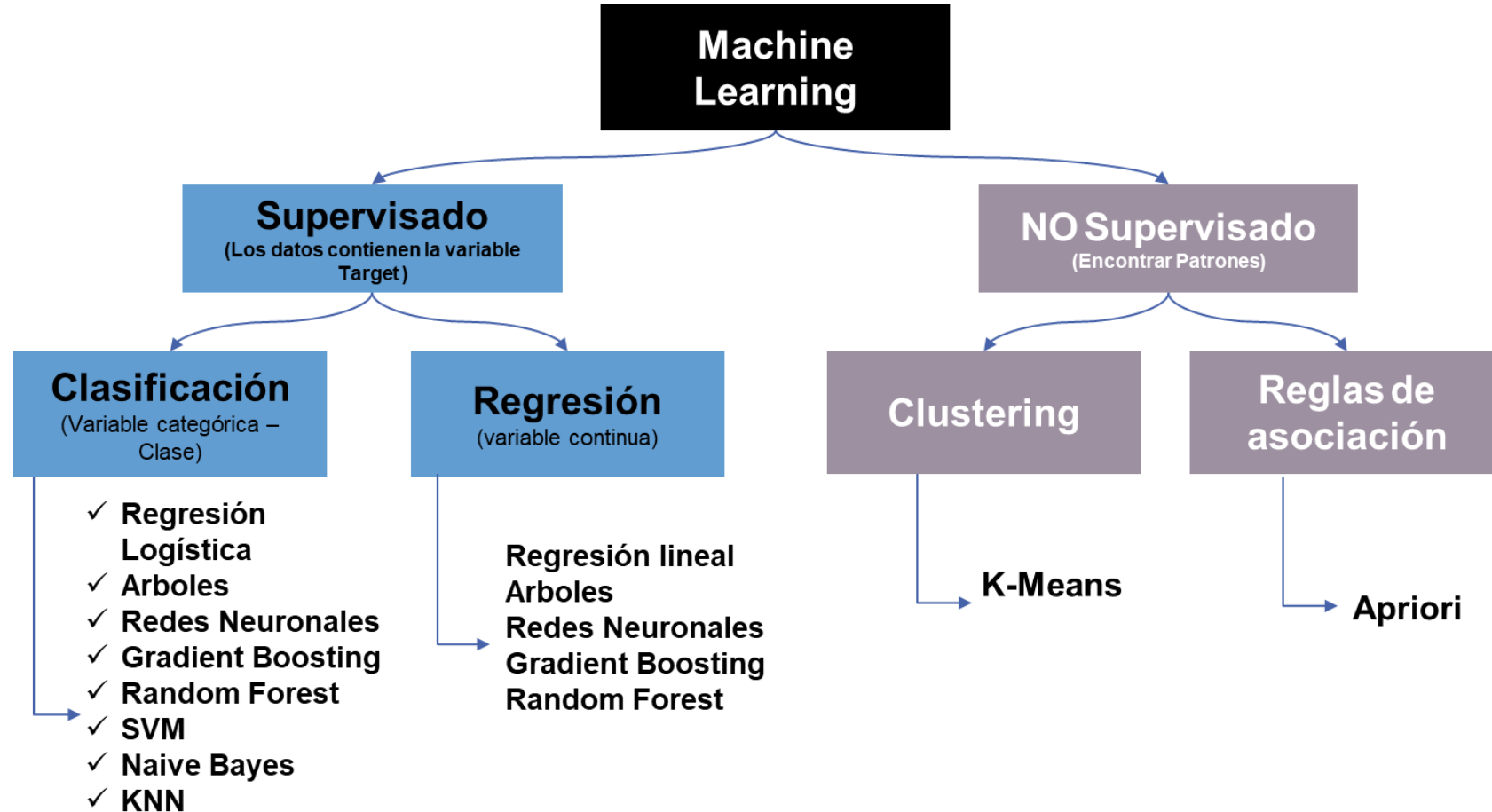
PROFESOR PABLO SCIOLLA





KNN

Para cada técnica de aprendizaje, hay modelos ya estudiados y desarrollados en diversas librerías que realizan la tarea.
Por ejemplo,



No free lunch theorem

- Nuestro modelo es una simplificación de la realidad
- La simplificación está basada en asumidos (sesgo del modelo)
- Los asumidos fallan en ciertas situaciones

Entonces, **“Ningún modelo funciona de forma optima en todas las situaciones”**

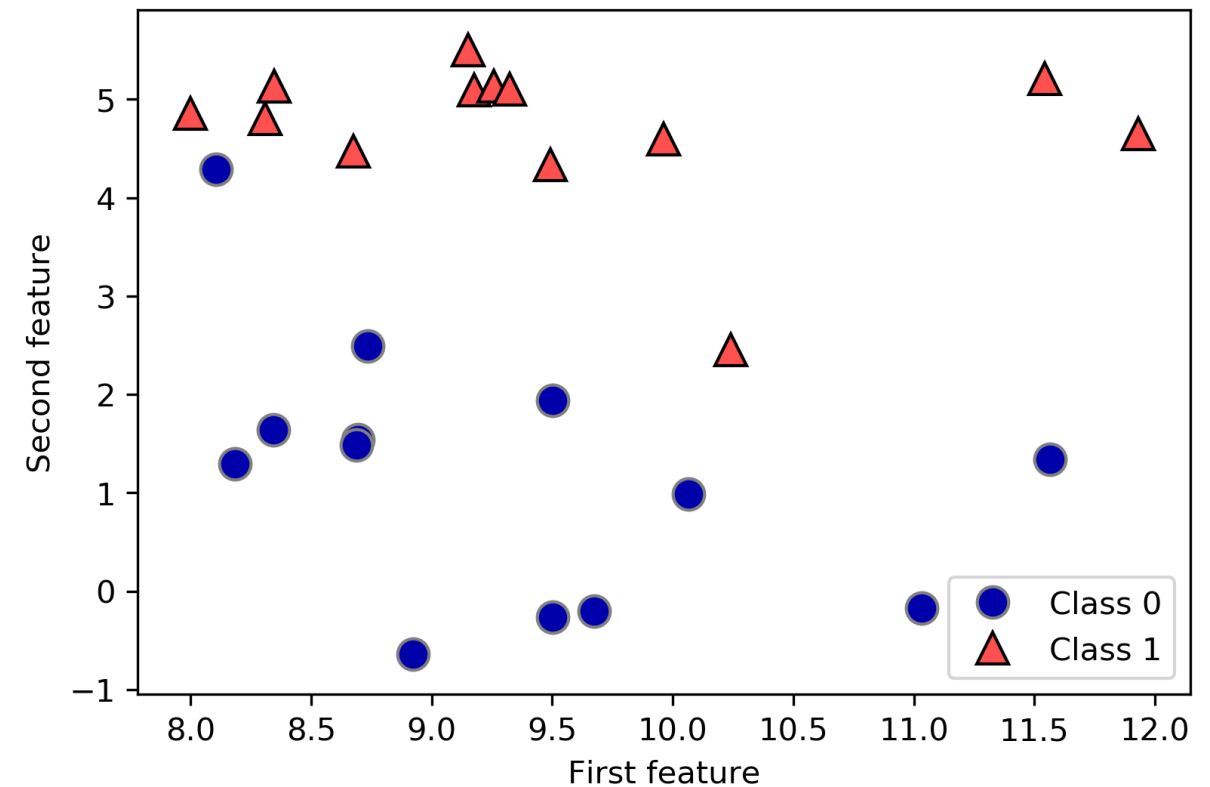
K-Vecinos más cercanos

KNN (1/6)

K-vecinos más cercanos o K-nearest neighbors (KNN), es un algoritmo del tipo de aprendizaje supervisado que tiene como objetivo la clasificación de datos

Conjunto de datos de dos características y dos clases

- Clase 0
- Clase 1



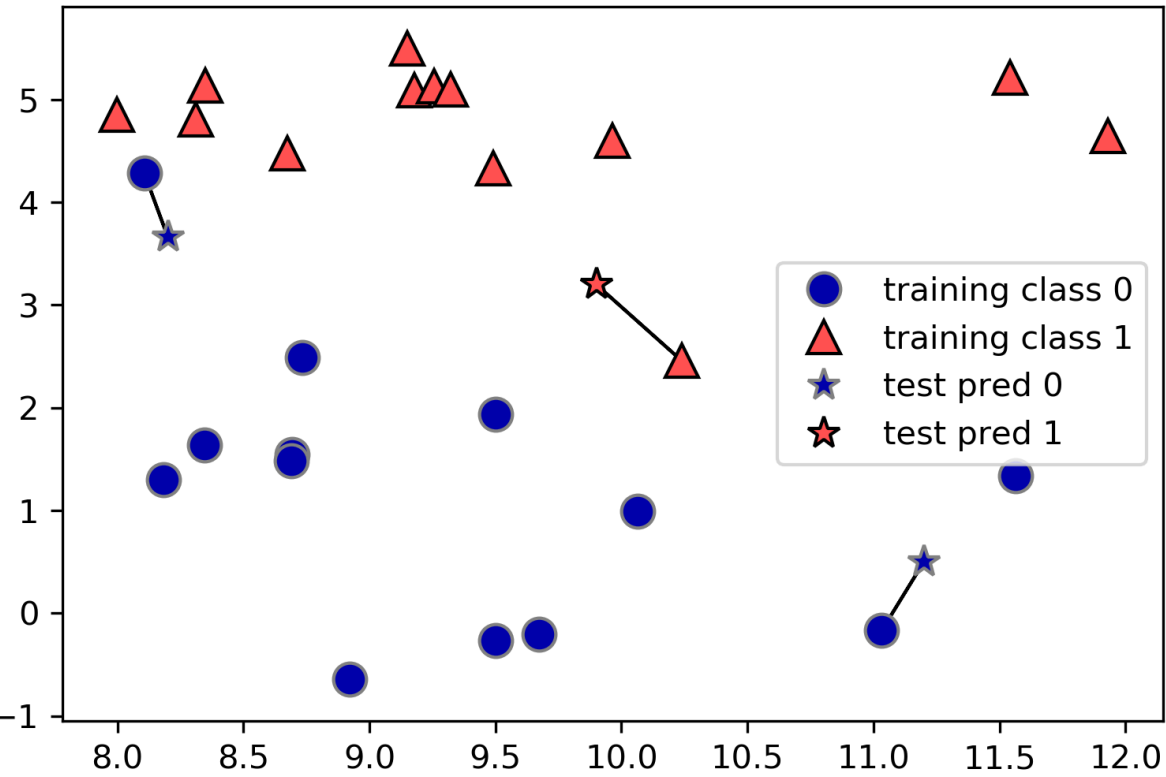
Müller, A., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*.

K-Vecinos más cercanos

KNN (2/6)

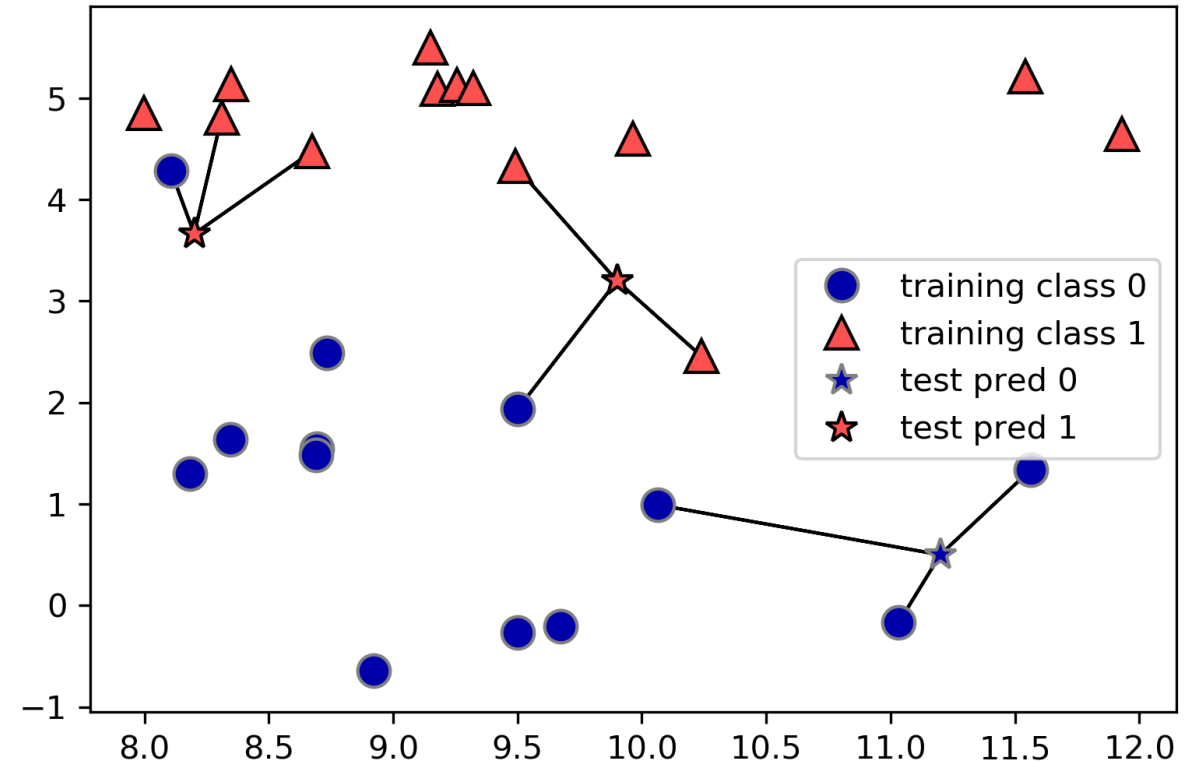
El algoritmo, a partir de un K, determina a que clase corresponde el nuevo dato. Asigna según cuantos k cercanos se agregan 3 nuevos puntos de datos marcados como estrellas.

K=1



Müller, A., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*.

K=3

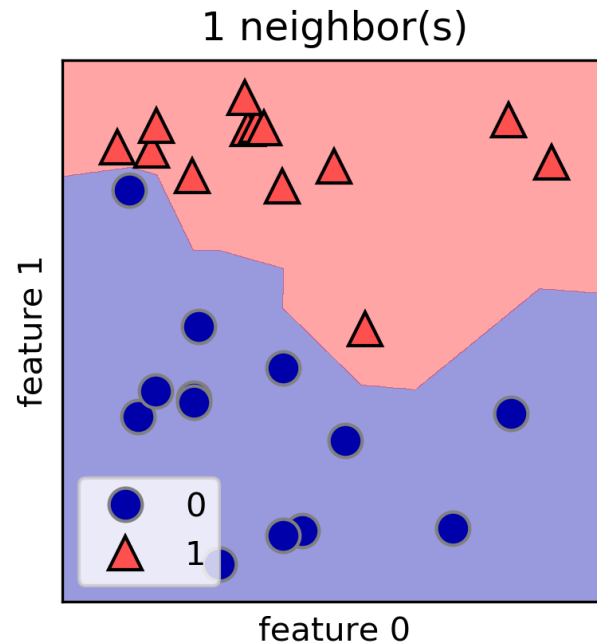


Cuando $k > 1$ se asigna por votación

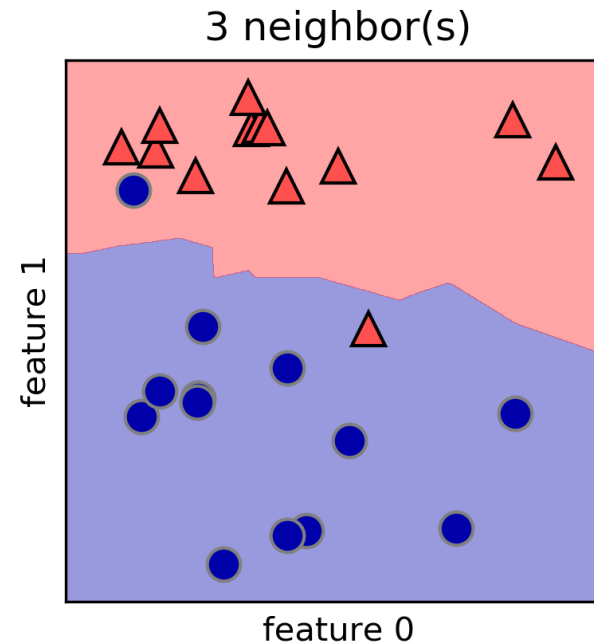
K-Vecinos más cercanos

KNN (3/6)

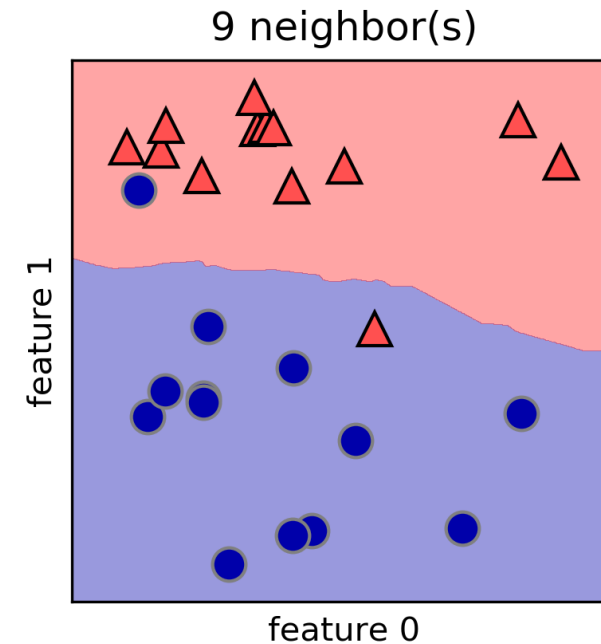
Fronteras de decisión con diferentes valores de K



Usar un solo vecino aproxima demasiado al set de datos de entrenamiento (el modelo es más complejo)



Incrementar K genera una frontera de decisión más suave, por lo que el modelo es más simple



Incrementar aún más K genera una frontera de decisión aún más suave.

Generalización, Overfitting y Underfitting

- **Modelo generalizable:** es un modelo que predicciones correctas en datos no vistos anteriormente. Se busca que los modelos sean generalizables.
- **Overfitting:** ocurre cuando se genera un modelo demasiado cercano a las particularidades del conjunto de datos de entrenamiento y se obtiene un modelo que funciona para esos datos pero no es generalizable a nuevos datos.
- **Underfitting:** ocurre cuando no se puede capturar toda la variabilidad de los datos y el modelo performa mal inclusive en el set de datos de entrenamiento.

Overfitting

Age	Number of cars owned	Owns house	Number of children	Marital status	Owns a dog	Bought a boat
66	1	yes	2	widowed	no	yes
52	2	yes	3	married	no	yes
22	0	no	0	married	yes	no
25	1	no	1	single	no	no
44	0	no	2	divorced	yes	no
39	1	yes	2	married	yes	no
26	1	no	2	single	no	no
40	3	yes	1	married	yes	no
53	2	yes	2	divorced	no	yes
64	2	yes	3	divorced	no	no
58	2	yes	2	married	yes	yes
33	1	no	1	single	no	no

Un modelo que hizo overfitting: Si el cliente tiene más de 45 años y tiene más de 3 hijos o no es divorciado entonces quiere comprar un bote. Es 100% exacto

Pero queremos saber si existen nuevos clientes que quieren comprar un bote, por lo que necesitamos una regla que trabaje bien para nuevos clientes, y tener un 100% de exactitud en el set de datos de entrenamiento no ayuda

Underfitting

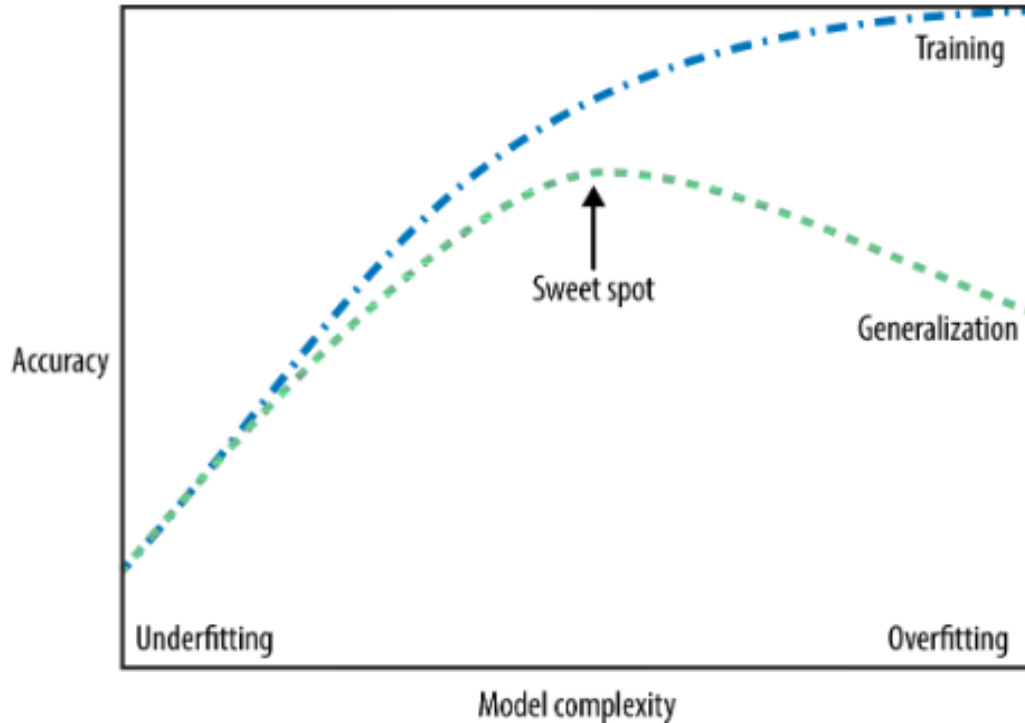
Age	Number of cars owned	Owns house	Number of children	Marital status	Owns a dog	Bought a boat
66	1	yes	2	widowed	no	yes
52	2	yes	3	married	no	yes
22	0	no	0	married	yes	no
25	1	no	1	single	no	no
44	0	no	2	divorced	yes	no
39	1	yes	2	married	yes	no
26	1	no	2	single	no	no
40	3	yes	1	married	yes	no
53	2	yes	2	divorced	no	yes
64	2	yes	3	divorced	no	no
58	2	yes	2	married	yes	yes
33	1	no	1	single	no	no

Un modelo que hizo underfitting: Todos los que tienen una casa quieren un bote

Basado en ese modelo, no predice bien ni siquiera en el conjunto de datos de entrenamiento (solo funciona en el 57% de los casos – 4/7)

Machine Learning

Generalización – Overfitting – Underfitting (4/4)



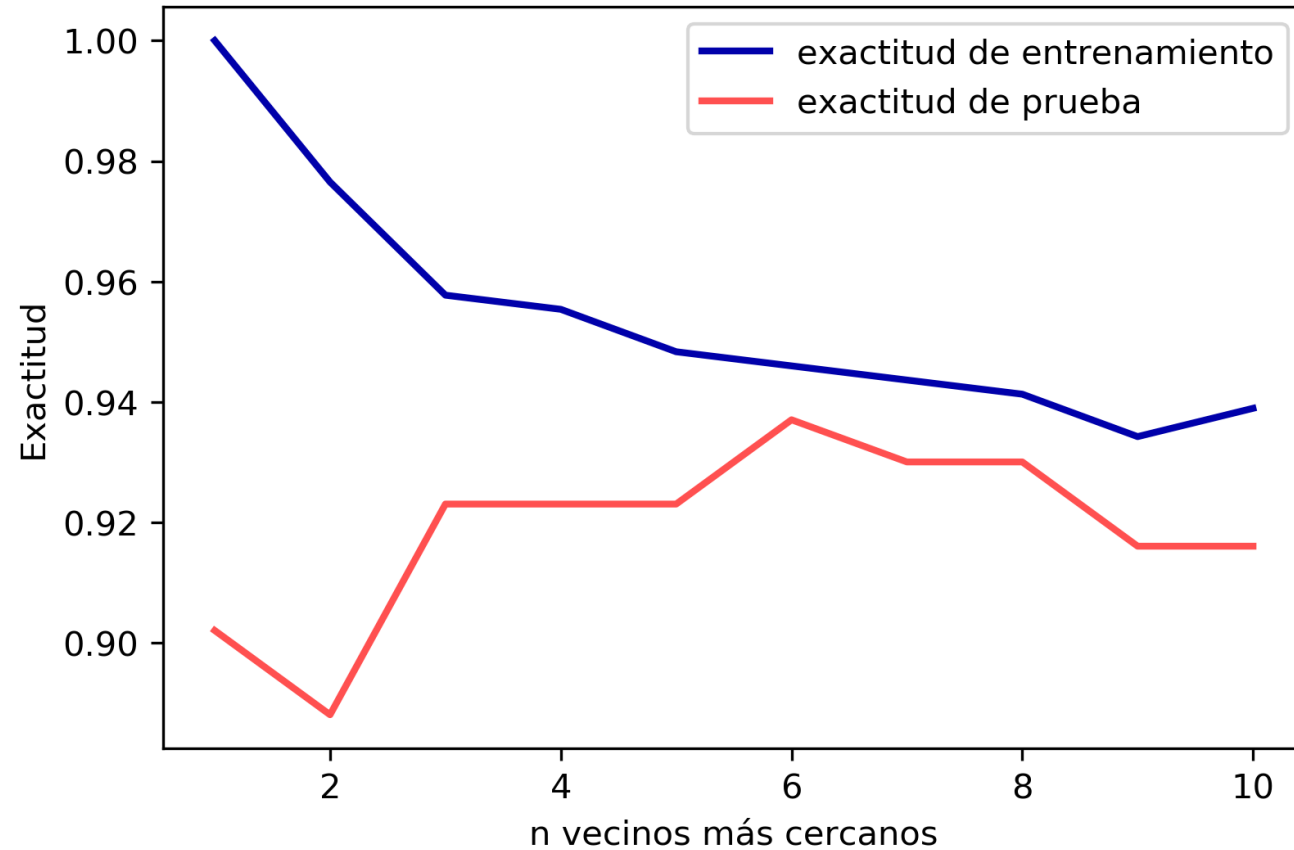
- Un modelo más complejo hará que sea más exacto sobre el conjunto de datos de entrenamiento.
- Sin embargo, si el modelo se vuelve demasiado complejo, se comienza a focalizar en cada punto de datos individual del conjunto de datos de entrenamiento y el modelo no generaliza bien a nuevos datos.
- Existe un punto intermedio entre complejidad y accuracy que permite una “buena” generalización sin caer en overfitting.

Müller, A., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*.

K-Vecinos más cercanos

KNN (4/6)

Se prueban modelos con diferentes K en un conjunto de datos

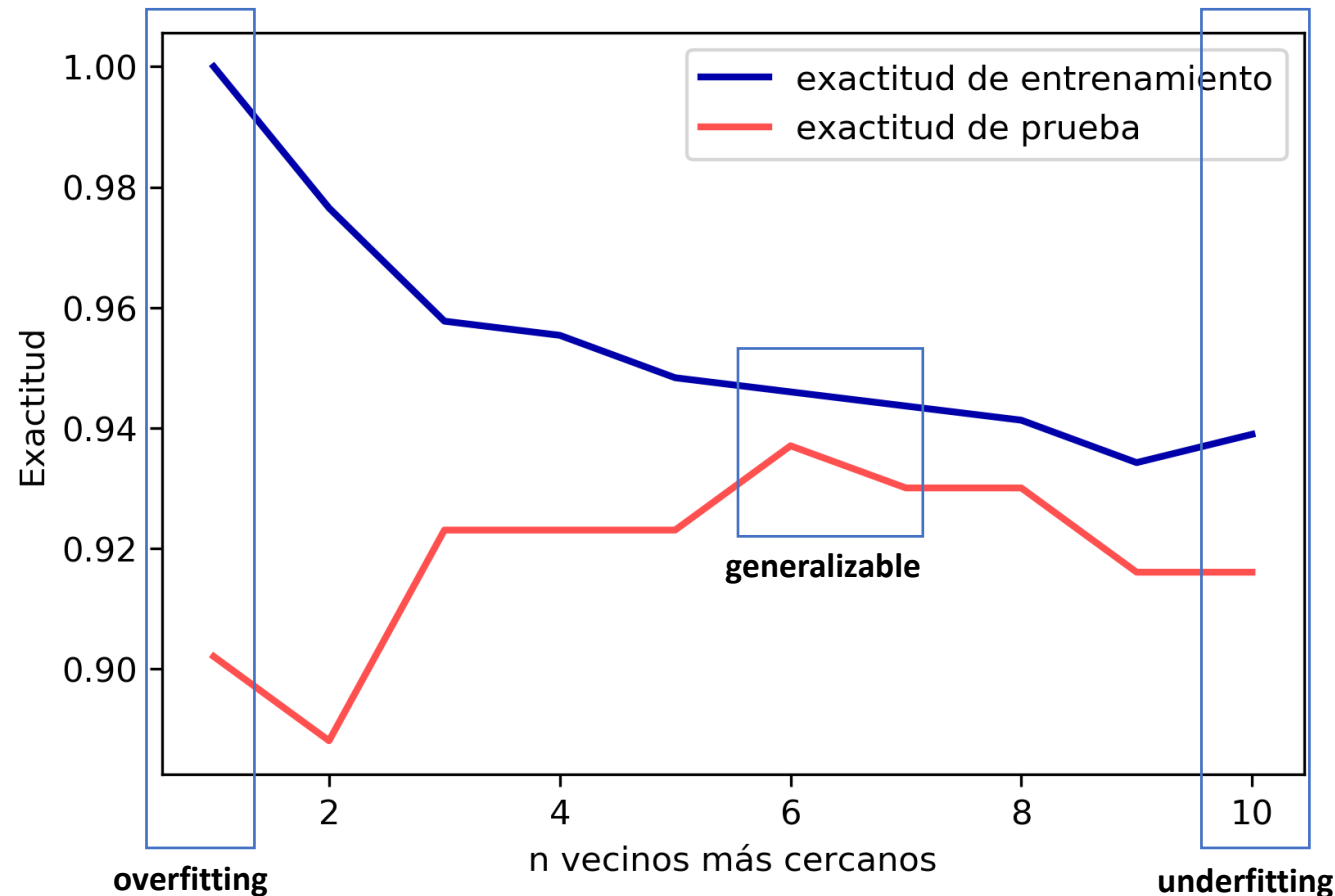


Müller, A., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*.

K-Vecinos más cercanos

KNN (5/6)

Se prueban modelos con diferentes K en un conjunto de datos



Buscamos encontrar el valor de K, donde el modelo es generalizable

K-Vecinos más cercanos

KNN (5/6)

En Python, con ayuda de Sklearn el código es como sigue

comando - ejemplo

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris_dataset = load_iris()

X_train, X_test, y_train, y_test =
train_test_split(iris_dataset['data'], iris_dataset['target'])

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train, y_train)

prediction = knn.predict(np.array([[5, 2.9, 1, 0.2]]))
```

Müller, A., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*.



Evaluación de modelos

Generalmente en clasificación binaria, se ordenan estas métricas en la matriz de confusión.

Matriz de Confusión:

- Permite visualizar el desempeño del modelo de clasificación binaria. Uno de sus veneficios es determinar si el modelo confunde dos clases, de ahí el nombre.

negative class	TN	FP
positive class	FN	TP
	predicted negative	predicted positive

true 'not nine'	401	2
true 'nine'	8	39
	predicted 'not nine'	predicted 'nine'

Ejemplo

TP = True Positive TN = True Negative FP = False Positive FN = False Negative

Evaluación De Modelos – Matriz De Confusión

Métricas para Clasificación – Matriz de confusión

real negativa (RN)	VN	FP
real positiva (RP)	FN	VP
	predicción negativa (PN)	predicción positiva (PP)

no aceptaron	401	2
aceptaron	8	39
	predicción no aceptar	predicción aceptar

$$\text{Exactitud (Accuracy)} = \frac{VP + VN}{VP + VN + FP + FN} = \frac{440}{450}$$

$$\text{Ratio Falsos Positivos (FPR)} = \frac{FP}{RN} = \frac{FP}{(FP+VN)} = \frac{2}{(2 + 401)}$$

$$\text{Precisión (VPP)} = \frac{VP}{PP} = \frac{VP}{(VP+FP)} = \frac{39}{(39+2)}$$

$$\text{Predictivo Negativo (VPN)} = \frac{VN}{PN} = \frac{VN}{(VN+FN)} = \frac{401}{(401 + 8)}$$

$$\text{Sensibilidad (Recall)} = \frac{VP}{RP} = \frac{VP}{(VP+FN)} = \frac{39}{(39+8)}$$

$$\text{Especificidad (SPC)} = 1 - \text{FPR} = \frac{VN}{(VN+FP)} = \frac{401}{(2+401)}$$

Accuracy, Precision y Sensibilidad

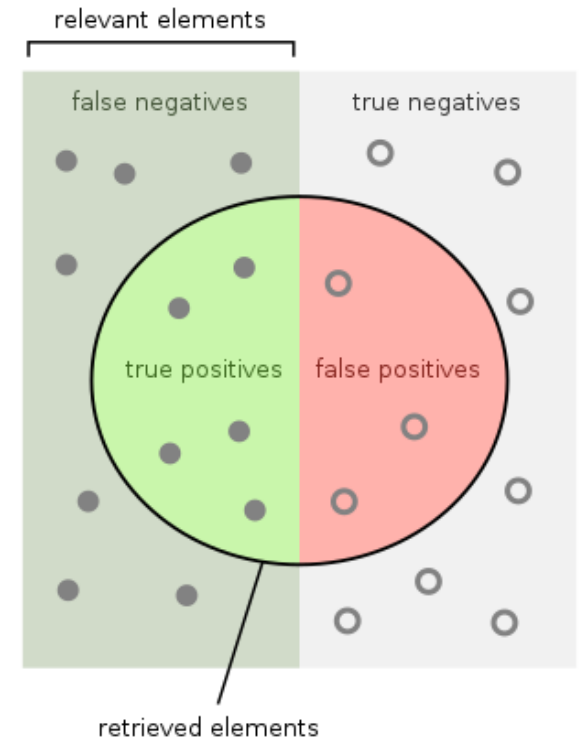
Métricas para Clasificación – Matriz de confusión

- **Exactitud (accuracy):** porcentaje de aciertos totales.
- **Sensibilidad (recall):** ¿qué porcentaje de los que aceptaron la promoción predijo que las iban a aceptar?
- **Precision:** qué porcentaje de veces que dije que el cliente iba a aceptar la promoción, efectivamente la aceptó.

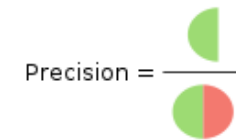
$$\frac{TP + VN}{VP + VN + FP + FN}$$

$$\frac{VP}{PP} = \frac{VP}{(VP+FP)}$$

$$\frac{VP}{RP} = \frac{VP}{(VP+FN)}$$

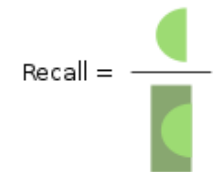


How many retrieved items are relevant?



Precision =

How many relevant items are retrieved?



Recall =

Precision and recall - Wikipedia

Es necesario contar con métricas de performance sobre los modelos desarrollados.
De esta manera es posible evaluar lo “bien” o “mal” que funcionan.

Dichas métricas dependen del tipo de modelo: regresión, clasificación, clustering, etc.

Métricas - Clasificación

Accuracy: exactitud

$$Accuracy = \frac{(VP + VN)}{VP + VN + FP + FN}$$

Precision: precisión, potencia predictiva positiva

$$Precision = \frac{VP}{(VP + FP)}$$

Busca limitar la cantidad de falsos positivos

Recall: sensibilidad

$$Recall = \frac{VP}{(VP + FN)}$$

Busca limitar la cantidad de falsos negativos,
identificando todos los casos positivos

F₁-score: medida balanceada de precision y recall

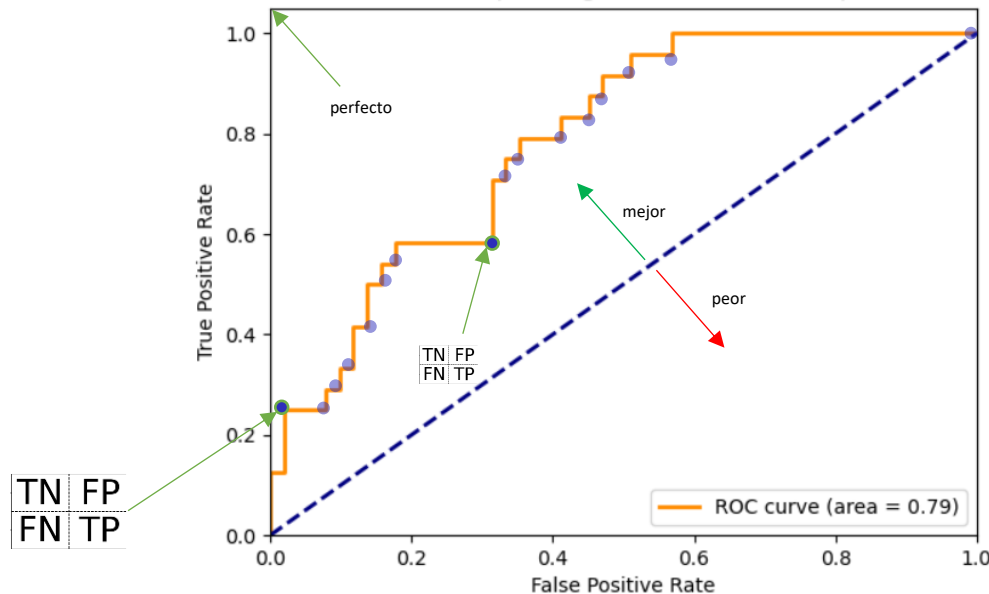
$$F_1 = 2 * \left(\frac{precision * recall}{precision + recall} \right)$$

TP = True Positive TN = True Negative FP = False Positive FN = False Negative

Una buena representación gráfica del desempeño de un modelo de clasificación es la curva ROC

Receiver Operation Characteristic (ROC)

- Es una representación gráfica de la sensibilidad de un modelo de clasificación binaria según varia el umbral de discriminación.
- Se representan los valores de False Positive Rate (FPR) vs. True Positive Rate (TPR) sobre el plano para distintos valores/modelos. Se conectan los puntos del plano.
- El área formada debajo de la curva simboliza el poder predictivo del modelo, independientemente de su especificación del momento
- Se dice que la línea diagonal es la base de cualquier modelo. El 50-50 del azar.



$$TPR = Precision = \frac{TP}{(TP + FP)}$$

$$FPR = 1 - Recall = \frac{FP}{(FP + TN)}$$

TP = True Positive TN = True Negative FP = False Positive FN = False Negative

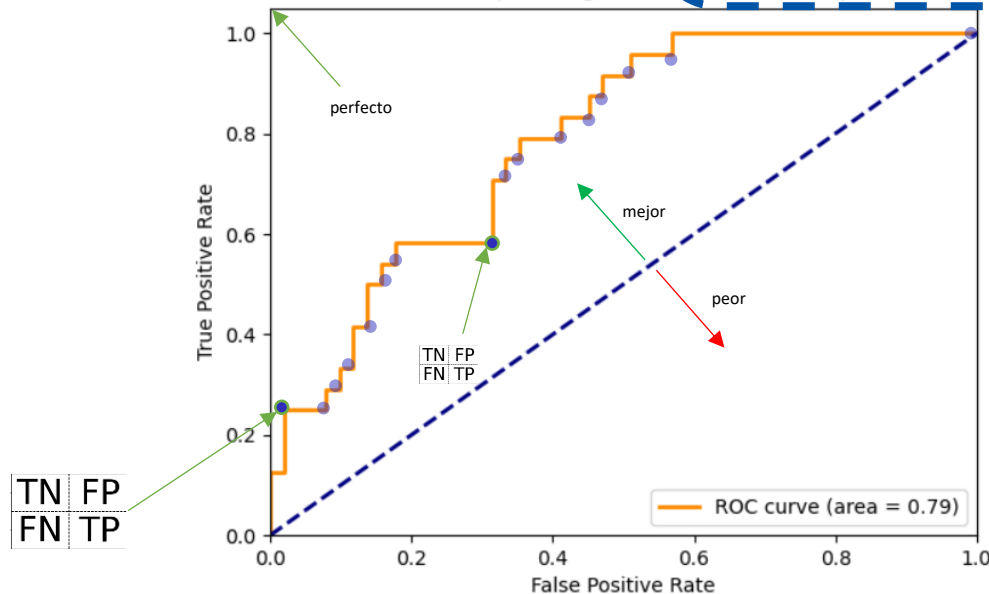
Una buena representación gráfica del desempeño de un modelo de clasificación es la curva ROC

comando - ejemplo

Receiver Operation Characteristic (ROC)

```
from sklearn.metrics import roc_curve, auc, roc_auc_score

iris = datasets.load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y)
classifier = model()
y_score = classifier.fit(X_train, y_train).score_function(X_test)
fpr, tpr, _ = roc_curve(y_test, y_score)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr)
```



$$TPR = Precision = \frac{TP}{(TP + FP)}$$

$$FPR = 1 - Recall = \frac{FP}{(FP + TN)}$$

TP = True Positive TN = True Negative FP = False Positive FN = False Negative

Métricas – Clasificación: en python

Accuracy

comando - ejemplo

```
from sklearn.metrics import accuracy_score
...
accuracy = accuracy_score(y_test, y_pred)
```

Precision

comando - ejemplo

```
from sklearn.metrics import precision_score
...
precision = precision_score(y_test, y_pred)
```

Recall

comando - ejemplo

```
from sklearn.metrics import recall_score
...
recall = recall_score(y_test, y_pred)
```

F1-Score

comando - ejemplo

```
from sklearn.metrics import f1_score
...
f1 = f1_score(y_test, y_pred)
```

Confusión Matrix

comando - ejemplo

```
from sklearn.metrics import confusion_matrix
...
cmatrix = confusion_matrix(y_test, y_pred)
```

Classification Report

comando - ejemplo

```
from sklearn.metrics import classification_report
...
report = classification_report(y_test, y_pred)
```

Demo Jupyter



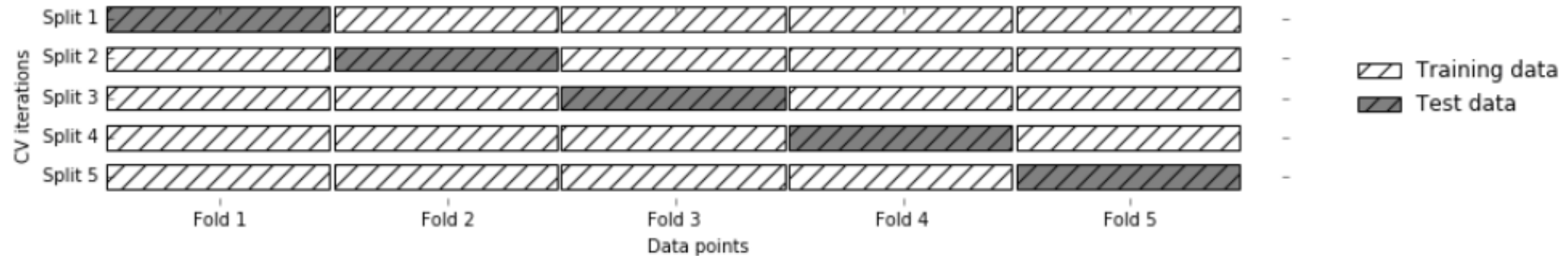
Evaluación de Modelos

Validación Cruzada – Cross Validation – (1/5)

- Cross validation es un método estadístico de evaluación de la performance de la generalización, que es más estable que usar un split en un training y test set
- En cross validation, los datos se particionan repetitivamente y varios modelos son entrenados.
- La versión más común es k-fold cross validation, donde k es un número especificado por el usuario, generalmente 5 o 10.

Evaluación de Modelos

Validación Cruzada – Cross Validation – (2/5)



- Por ejemplo, cuando se realiza 5-fold cross validation, los datos se parten en 5 partes de aproximadamente igual tamaño, llamados folds.
- El primer modelo se entrena utilizando el fold 1 como el set de prueba y los restantes (2,3,4,5) se usan como sets de entrenamiento: el modelo se construye con los datos (2-5) y el accuracy se evalúa sobre 1.
- Entonces se construye un segundo modelo utilizando fold 2 como el set de prueba y los datos en folds (1,3,4,5) como sets de entrenamiento.

Ventajas

- Recordar que `train_test_split` realiza un particionamiento aleatorio de los datos. Se puede tener suerte y encontrar un data set representativo, pero quizás esto no pase. Cuando se usa cross validation, cada muestra va a estar en el set de datos de prueba exactamente una vez. De esta manera el modelo necesita generalizar bien sobre todas las muestras del dataset para que cross validation devuelva un buen accuracy.
- Tener múltiples splits de los datos proporciona información acerca de que tan sensible es el modelo a la selección del training set.
- En Cross Validation, se usan los datos más eficientemente. Cuando se usa `train_test_split` se utiliza por lo general el 75% de los datos para training y 25% para test. Usando Cross Validation con 5 folds, se usa el 80% de los datos para entrenamiento. Con 10 folds, se usa 90%.

Desventajas

- Costo computacional. Como se entrenan k modelos en lugar de un único modelo, el proceso va a ser k veces más lento que hacer un split simple de los datos.